Backend API Report – Project V Server

1. Authentication Flow (User)

Endpoints

- **Login (send OTP)**
  - `POST http://103.172.252.35/api/auth/user/login`
  - **Body:**

```
{
  "email": "san2722soni@gmail.com"
}
```

  - **Response:**

```
{
  "OTP": "003221"   // Suppressed in production; only for testing/dev
}
```

  - **Logic:**
    - Generates OTP via `generateOtp()`.
    - Deletes old OTP entry if exists.
    - Stores new OTP in `UserOtp` collection.
    - Master OTP = `1247` (works even if DB OTP is different).
    - Currently returns OTP (for debug). Suppress before production.

- **Verify OTP**
  - `POST http://103.172.252.35/api/auth/user/verifyOtp`

- **Body:**

```
{
  "email": "san2722soni@gmail.com",
  "otp": "003221"
}
```

- **Response:**

```
{
  "refreshToken": "9d22edf9591d44f99bd2e68e1c88baaf",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

- **Logic:**
  - Validates OTP (either master = `1247` or DB OTP).
  - Deletes OTP after use (no reuse).
  - If new user → creates user with tokens.
  - If existing user → checks token expiry, refreshes if needed.
  - Returns **JWT access token** + **refresh token**.

Database Models

- **User**
  - `email`
  - `refreshToken`
  - `tokenEAT` (expiry timestamp)
  - `isUser`
- **Otp**
  - `email`
  - `value` (OTP)

2. Game Info Flow

Characters API

- **Get Active Characters**
  - `GET http://103.172.252.35/api/game/info/characters`
  - **Headers (Required):**

    ```
    Authorization: Bearer <accessToken>
    ```

  - **Response Example:**

    ```
    [
     {
      "_id": "68c53b0a03cbac194c93dafe",
      "health": 100,
      "maxHealth": 100,
      "isActive": true,
      "isStun": false,
      "tribe": "Test_Tribe_UPDATED",
      "availableRole": ["General", "Warden"]
     }
    ]
    ```

  - **Logic:**
    - Validates JWT accessToken.
    - Returns all characters where `isActive = true`.
- **Get Active Character by ID**
  - `GET http://103.172.252.35/api/game/info/characters/:id`
  - **Logic:**
    - Validates `id` format (`mongoose.Types.ObjectId`).

- Returns single active character if found, else throws error.

---

3. Security Notes

- ✅ OTP auto-deletes after use (prevents replay).
- ✅ Master OTP (`1247`) for dev testing.
- ⚠️ Currently OTP is being **returned in response** during login (must be removed in production).
- ✅ Access/Refresh tokens are issued via JWT + random string.
- ✅ Authorization required for game info routes.

Matchmaking API

This module handles player matchmaking using **Redis** for queueing and **MongoDB** for persistent game state. It also relies on cron jobs to automatically clean inactive players and stale games.

Endpoints

1. `POST /api/matchmaking/findAMatch`

Description

Adds the user to the matchmaking queue. If 4 players are available, creates a new game.

Headers

```
Authorization: Bearer <JWT_TOKEN>
```

Example Request

```
POST /api/matchmaking/findAMatch
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response (added to queue)

```
{
  "matchFound": false,
  "message": "Player added to queue",
  "queueLength": 2
}
```

Example Response (already in queue)

```
{
  "matchFound": false,
  "message": "Player is already in a queue"
}
```

Example Response (game created)

```
{
  "matchFound": true,
  "game": {
    "_id": "66f8d3c91538e95dc7af83e1",
    "saltKey": "",
    "players": [
      { "userId": "101", "secret": "", "characterId": "" },
      { "userId": "102", "secret": "", "characterId": "" },
      { "userId": "103", "secret": "", "characterId": "" },
      { "userId": "104", "secret": "", "characterId": "" }
    ]
  }
}
```

---

2. POST /api/matchmaking/cancelSearch

Description

Removes the player from the matchmaking queue.

Headers

```
Authorization: Bearer <JWT_TOKEN>
```

Example Request

```
POST /api/matchmaking/cancelSearch
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response

```
{
  "removed": true
}
```

---

3. GET /api/matchmaking/getStatus

Description

Returns the user's current status (in game or in queue). Also updates the user's heartbeat.

Headers

```
Authorization: Bearer <JWT_TOKEN>
```

Example Request

```
GET /api/matchmaking/getStatus
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response (in queue)

```
{
  "inGame": false,
```

```
  "inQueue": true
}
```

Example Response (in game)

```
{
  "inGame": true,
  "gameId": "66f8d3c91538e95dc7af83e1",
  "playerId": "101",
  "saltKey": ""
}
```

Example Response (idle)

```
{
  "inGame": false,
  "inQueue": false
}
```

Cron Jobs

1. Cleanup Inactive Players

- Runs every **1 minute**.
- Removes players from Redis queue if they haven't sent a heartbeat in 60s.

2. Cleanup Stale Games

- Runs every **5 minutes**.
- Deletes games older than 5 minutes (where players never proceeded).

Flow Example

1. User calls `POST /findAMatch` → enters queue.
2. User's frontend polls `GET /getStatus` every ~5s:

- Keeps them alive (heartbeat updated in Redis).
- Tells if they got matched.

3. If user cancels → `POST /cancelSearch`.
4. If user disconnects → cron job auto-cleans them.
5. When 4 players ready → new game created in MongoDB.

Notes

- All endpoints require **JWT Authentication**.
- Queue is **FIFO** in Redis.
- Cron jobs ensure stale data never clogs the system.